

Formal Models of Language

The 80/20 Revision Guide

Part IB — Cambridge CST — Paper 7

How this guide is organised

Formal Models of Language sets one or two questions a year on **Paper 7**. Across **eight years of past papers (2018–2025, 15 questions)**, the tagged topic frequencies are:

Topic	Appearances	Share
Context-free grammars & parsing (Earley, parse trees)	7	47%
Formal-language theory & automata (Chomsky hierarchy)	4	27%
Entropy & information theory	3	20%
Noisy-channel model (NLP)	3	20%
Syntax trees & tree-adjointing grammars	3	20%
Formal vs. natural language	2	13%
Regular expressions & finite automata	2	13%
Grammar induction (Gold's theorem)	1	7%
Dependency parsing & grammars	1	7%
Categorical / lexicalised grammars	1	7%
Statistical language models	1	7%
Complexity & human language processing	1	7%

80/20 verdict. Two clusters dominate. **(1) Grammars, the Chomsky hierarchy and parsing** (CFGs, pumping lemmas, Earley, dependency/TAG/categorical) account for well over half the marks. **(2) Information theory** (entropy, mutual information, the noisy channel) is the other reliable question, $\sim 40\%$ across its tags. Together they are $\sim 90\%$ of the paper. *Vector-space models / word embeddings have **never** been examined (0/15)* — a single paragraph of insurance is enough.

This document is ordered that way: **grammars & parsing first, then information theory, then the long tail.**

1 Grammars, the Chomsky hierarchy & parsing (Tier 1)

1.1 Phrase-structure grammars and the hierarchy

Exam-critical

A **phrase-structure (generative) grammar** is a tuple $G = (\mathcal{N}, \Sigma, S, \mathcal{P})$: non-terminals $\mathcal{N}$, terminals Σ (with $\mathcal{N} \cap \Sigma = \emptyset$), start symbol $S \in \mathcal{N}$, and production rules $\mathcal{P}$ of the form $w \rightarrow v$ with $w \in (\Sigma \cup \mathcal{N})^* \mathcal{N} (\Sigma \cup \mathcal{N})^*$ (at least one non-terminal on the left).

A **derivation step** $w \Rightarrow v$ rewrites one rule's LHS; $\overset{*}{\Rightarrow}$ is its reflexive-transitive closure, and $\mathcal{L}(G) = \{w \in \Sigma^* \mid S \overset{*}{\Rightarrow} w\}$.

The Chomsky hierarchy groups grammars by the shape of their rules; each type is a strict superset of the one above in expressive power, recognised by a more powerful machine.

Type	Class	Rule form	Machine	Recognition
3	regular	$A \rightarrow aB \mid a$ (or $A \rightarrow Ba$)	DFA / NFA	$O(n)$
2	context-free	$A \rightarrow \alpha, \alpha \in (\mathcal{N} \cup \Sigma)^*$	push-down automaton	$O(n^c)$
1	context-sensitive	$\alpha A \beta \rightarrow \alpha \gamma \beta, \gamma \neq \epsilon$	linear-bounded automaton	$O(c^n)$
0	recursively enum.	$\alpha \rightarrow \beta$ (any)	Turing machine	undecidable

Worth knowing

Closure properties (examinable as one-liners). Every Chomsky class is closed under *union* ($\mathcal{L}(G_1) \cup \mathcal{L}(G_2)$ is the same type). Every class is closed under *intersection with a regular language* (regular $\cap$ CF = CF). **Weak vs. strong equivalence:** two grammars are *weakly* equivalent if they generate the same string set; *strongly* equivalent if they also assign the same tree structures.

Machines (one line each). A **DFA** $M = (\mathcal{Q}, \Sigma, \Delta, s, \mathcal{F})$ accepts the regular languages; transition $\delta : \mathcal{Q} \times \Sigma \rightarrow \mathcal{Q}$. A **push-down automaton** adds a LIFO stack (δ reads state, input *and* stack-top; transitions written $a : A/B = \text{on } a, \text{ pop } A, \text{ push } B$) and accepts exactly the context-free languages. A **linear-bounded automaton** (bounded tape) gives context-sensitive; an unrestricted **Turing machine** gives recursively enumerable.

1.2 Pumping lemmas — the standard “prove it’s not...” tool

Exam-critical

Regular pumping lemma. If $\mathcal{L}$ is regular then $\exists l$ s.t. every $w \in \mathcal{L}$ with $|w| \geq l$ can be written $w = u_1 v u_2$ with $|v| \geq 1$, $|u_1 v| \leq l$, and $u_1 v^n u_2 \in \mathcal{L}$ for all $n \geq 0$.

Context-free pumping lemma. If $\mathcal{L}$ is context-free then $\exists k$ s.t. every $w \in \mathcal{L}$ with $|w| \geq k$ splits as $w = u_1 v u_2 y u_3$ with $|vy| \geq 1$, $|vu_2 y| \leq k$, and $u_1 v^n u_2 y^n u_3 \in \mathcal{L}$ for all $n \geq 0$. (Two pumped regions, pumped *together*.)

Proof template (do it as a game). To show $\mathcal{L}$ is *not* regular: for every l , *you* pick a witness $w \in \mathcal{L}$ with $|w| \geq l$; the adversary splits it as $u_1 v u_2$ respecting the constraints; *you* pick an n (usually $n = 0$ or 2) making $u_1 v^n u_2 \notin \mathcal{L}$.

Worked: $a^n b^n$ is not regular

Pick $w = a^l b^l$. Any split with $|u_1 v| \leq l$ forces $v = a^r, r \geq 1$. Then $u_1 v^2 u_2 = a^{l+r} b^l \notin \mathcal{L}$. For *not context-free*, the canonical witness is $a^n b^n c^n$ with the CF lemma (v, y can cover at most two of the three letters, so pumping unbalances the third).

Gotcha

The pumping lemma is a **necessary, not sufficient** condition — you can only use it to prove a language is *not* in a class, never to prove it *is*. And the adversary chooses the split: your proof must work for *every* legal $u_1 v u_2$, so keep the witness general (use the bound $|u_1 v| \leq l$ to pin v inside the a -block).

1.3 Parsing — the single most examined topic

Two meanings of “parse.” *Recognition:* decide whether $w \in \mathcal{L}(G)$ (complexity per the hierarchy table). *Structure:* recover the most useful constituent (phrase-structure) tree — the usual NLP task. **Chomsky Normal Form** ($A \rightarrow BC$ or $A \rightarrow a$) is the precondition assumed by several parsing algorithms; every CFG has a weakly-equivalent CNF.

Exam-critical

The Earley parser is the headline algorithm. It is a top-down, dynamic-programming chart parser, $O(n^3)$, that handles *any* CFG (including ambiguous ones) and recovers *all* derivations.

State is a **dotted rule** $A \rightarrow \alpha \bullet \beta [i, j]$: rule $A \rightarrow \alpha \beta$, dot marking how much is parsed, spanning input positions i to j . Initialise with $S \rightarrow \bullet \alpha [0, 0]$; accept when $S \rightarrow \alpha \bullet [0, n]$ is in the chart.

The three operations, applied until the chart stops growing:

- **Predict:** from $A \rightarrow \alpha \bullet B \beta [i, j]$ and rule $B \rightarrow \gamma$, add $B \rightarrow \bullet \gamma [j, j]$ (expand the next non-terminal top-down).
- **Scan:** from $A \rightarrow \alpha \bullet a [i, j - 1]$ with input $a_j = a$, add $A \rightarrow \alpha a \bullet [i, j]$ (consume a terminal).
- **Complete:** from a finished $B \rightarrow \gamma \bullet [k, j]$ and a waiting $A \rightarrow \alpha \bullet B \beta [i, k]$, add $A \rightarrow \alpha B \bullet \beta [i, j]$ (a sub-tree is done; advance its parent).

Record a **history** per edge so the tree(s) can be reconstructed; shared sub-trees and **ambiguity packing** keep it polynomial despite exponentially many trees.

Worth knowing

Shift-reduce / LR(k) parsers (from Compiler Construction) recognise *deterministic* languages with one analysis: SHIFT a token onto the stack, REDUCE when the stack top matches a rule's RHS. They cannot cope with the inherent ambiguity of natural language without backtracking — which is exactly why Earley (or a probabilistic parser) is used for NL.

1.4 Dependency grammars

A **dependency tree** has the words themselves as nodes and labelled *head* $\rightarrow$ *dependent* edges (e.g. nsubj, dobj, nmod). A valid tree is rooted and connected; **projective** if it can be drawn with no crossing edges. Formally $G_{dep} = (\Sigma, \mathcal{D}, s, \perp, \mathcal{P})$ with direction markers $\mathcal{D} = \{\mathcal{L}, \mathcal{R}\}$.

Worth knowing

Shift-reduce dependency parsing. Replace REDUCE with LEFT-ARC / RIGHT-ARC, which combine the top two stack items, keep the *head*, and record the dependency. In practice it is **grammarless / data-driven**: a classifier (features from stack+buffer, often word embeddings) picks the action, trained on a **dependency bank**; beam search avoids early wrong commitments. *Projective* dependency grammars are weakly equivalent to CFGs (map via a notion of *headedness*).

1.5 Tree-adjoining & categorial grammars (mildly context-sensitive)

Worth knowing

Tree-adjoining grammar (TAG) rewrites *trees*, not symbols: a set of **initial** (α) **trees** and **auxiliary** (β) **trees**, combined by **substitution** (replace a non-terminal leaf with an α -tree) and **adjunction** (splice a β -tree, whose root and *foot* are the same non-terminal, into an internal node). Parsing is $O(n^6)$. TAG languages sit strictly between context-free and context-sensitive — the **mildly context-sensitive** class — argued to be just expressive enough for natural language (e.g. cross-serial dependencies).

Categorial grammar is lexicalised: each word carries *types* built from primitives with $\backslash$ and $/$; parsing is type combination via **forward application** ($A/B \quad B \Rightarrow A$) and **backward application** ($B \quad A \backslash B \Rightarrow A$). Classic CG = CFG in power; **combinatory** CG (adds composition

+ type-raising) reaches the mildly context-sensitive class.

2 Information theory & language (Tier 1)

Exam-critical

Entropy of a source X over alphabet $\mathcal{X}$ (average information / bits per message):

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x) \quad (\text{uniform: } H = \log_2 M).$$

Surprisal of one message: $s(x) = -\log_2 p(x)$ — rare events carry more information; entropy is the average surprisal.

Worth knowing

The whole toolkit (state the formula, then interpret).

$$\text{Conditional entropy : } H(Y | X) = -\sum_{x,y} p(x,y) \log_2 p(y | x)$$

$$\text{Joint entropy : } H(X, Y) = -\sum_{x,y} p(x,y) \log_2 p(x,y)$$

$$\text{Chain rule : } H(X, Y) = H(X) + H(Y | X)$$

$$\text{Mutual information : } I(X; Y) = H(X) - H(X | Y) = H(Y) - H(Y | X)$$

$$\text{Relative entropy (KL) : } D(p \| q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)}$$

$$\text{Cross-entropy : } H(X, q) = H(X) + D(p \| q) = -\sum_x p(x) \log_2 q(x)$$

$$\text{Entropy rate : } H_{\text{rate}}(L) = \lim_{n \rightarrow \infty} \frac{1}{n} H(X_1 \dots X_n)$$

Mutual information is the reduction in uncertainty about X from knowing Y ; equivalently $I(X; Y) = D(p(x, y) \| p(x)p(y))$ — distance from independence.

Exam-critical

Noisy-channel model. A channel $p(y | x)$ maps input to output; **capacity** $C = \max_{p(X)} I(X; Y)$ is the max error-free rate. The **NLP noisy-channel framework**: recover the most likely input given the observed output via Bayes,

$$\hat{i} = \arg \max_i p(i | o) = \arg \max_i \underbrace{p(o | i)}_{\text{channel model}} \underbrace{p(i)}_{\text{language model}}.$$

$p(i)$ is usually an **n -gram** model (an n -gram conditions on the previous $n-1$ items; estimate by MLE $p_{\text{mle}}(w_1 \dots w_n) = \frac{\text{count}(w_1 \dots w_n)}{\text{count}(w_1 \dots w_{n-1})}$). Applications: spelling correction, speech recognition, machine translation.

Worth knowing

Information theory $\rightarrow$ **language theories** (the “why” the course cares). *Is language an efficient code?* word lengths track frequency / surprisal (Zipf). *Uniform Information Density*: speakers spread information evenly across the signal, choosing variants that avoid spikes/troughs

in surprisal. *Ambiguity* can be beneficial when context disambiguates — an efficient code reuses short forms.

3 The long tail (Tier 2/3)

Formal vs. natural language (2/15). Natural language is *ambiguous*, *productive/recursive*, and shows *displacement*; formal languages are exact subsets of Σ^* . Key distinctions to deploy: **competence vs. performance**; **grammaticality vs. statistical frequency**; **weak vs. strong generative capacity**. The standard argument that NL is *not* context-free is **cross-serial dependencies** (Swiss-German / Dutch), motivating mildly context-sensitive grammars (TAG/CCG).

Grammar induction (1/15). **Gold's theorem** (identification in the limit): no *superfinite* class of languages (all finite languages plus at least one infinite one — hence the regular languages and above) is learnable from *positive examples alone*. Consequences: learners need negative evidence, a prior/bias, or statistical/distributional cues. *Learnable* classes and *categorial-grammar* induction are the positive side.

Vector-space models (0/15 — insurance only). Words as vectors under the **distributional hypothesis** (“meaning = company a word keeps”); **word2vec** (skip-gram / CBOW) learns dense embeddings; cosine similarity measures semantic relatedness. Newer syllabus content — know the one-paragraph gist.

Exam checklist

Exam-critical

1. **Grammar & hierarchy:** define $G = (\mathcal{N}, \Sigma, S, \mathcal{P})$; reproduce the Type 0–3 rule-form table and its machines (DFA/PDA/LBA/TM) cold.
2. **Pumping:** state *both* lemmas; run the $a^n b^n$ (regular) and $a^n b^n c^n$ (CF) proofs as an adversary game. Remember: necessary, not sufficient.
3. **Earley:** dotted rule $A \rightarrow \alpha \bullet \beta [i, j]$; the three steps **predict** / **scan** / **complete**; $O(n^3)$; recovers all trees via edge histories. Be ready to run it by hand on a 3–4 word sentence.
4. **Dependency/TAG/categorial:** head→dependent trees + shift-reduce (left/right-arc, data-driven); TAG substitution/adjunction = mildly context-sensitive ($O(n^6)$); categorial forward/backward application.
5. **Information theory:** $H(X) = -\sum p \log_2 p$, surprisal $-\log_2 p(x)$, $I(X; Y) = H(X) - H(X | Y)$, chain rule, KL/cross-entropy. Noisy channel: $\hat{i} = \arg \max_i p(o | i)p(i)$ with an n -gram $p(i)$.
6. **Theory link:** UID, Zipf, ambiguity-as-efficiency; Gold's theorem (no superfinite class learnable from positive data only); cross-serial dependencies $\Rightarrow$ NL is not context-free.